Sum The File Project Report

a. I decided to tackle this problem first by breaking into the desired function I needed my code to perform. I figured I needed to first open the file and create a starting sum variable, initialized to 0. Then I would use a loop that did 3 specific functions. The first function would be to read in the integer on the first line. Next would be to convert the string version of the integer into an actual integer. The third would be to add this value to our sum variable and go to the next line. Once the entirety of the file has been read, I would close the file, print the sum, and exit code.

b. Code:

```
extern printf

section .data
    filename    db "randomInt100.txt", 0  ; File name
    filemode    equ 0                     ; Read-only
    buffer_size equ 1024                  ; Buffer size
    fmt_sum     db "Sum: %d", 10, 0       ; Format

section .bss
    buffer      resb buffer_size          ; Buffer to store file content
    sum         resd 1                    ; Total Sum

section .text
    global _start

_start:
    ; Initialize sum to 0
    mov dword [sum], 0

    ; Open file
    mov eax, 5          ; sys_open
    mov ebx, filename    ; File name
    mov ecx, filemode
    mov edx, 0
    int 0x80            ; Call kernel
    cmp eax, 0
    js exit             ; If error, exit
    mov edi, eax        ; Store file descriptor in edi

    ; Read from file
```

```
    mov eax, 3          ; sys_read
    mov ebx, edi        ; File descriptor
    mov ecx, buffer     ; Buffer to store data
    mov edx, buffer_size ; Bytes to read
    int 0x80
    cmp eax, 0
    jle close_file      ; If error or EOF, close file
    mov esi, eax        ; Store number of bytes read in esi

    ; Parse and sum integers
    xor edx, edx        ; Index into buffer
    xor ecx, ecx        ; Current integer being parsed

parse_loop:
    cmp edx, esi        ; Check if we've reached the end of the buffer
    jge finalize_sum    ; If yes, finalize the sum and exit
    movzx eax, byte [buffer + edx] ; Load the next character
    inc edx             ; Move to the next character
    cmp al, 10          ; Check for newline (ASCII 10)
    je add_to_sum
    cmp al, '0'
    jl skip_char        ; If less than '0', skip
    cmp al, '9'
    jg skip_char        ; If greater than '9', skip
    sub al, '0'         ; Convert ASCII to integer
    imul ecx, ecx, 10   ; Multiply current number by 10
    add ecx, eax        ; Add new digit to the current number
    jmp parse_loop

skip_char:
    jmp parse_loop

add_to_sum:
    add dword [sum], ecx ; Add the current number to the sum
    xor ecx, ecx        ; Reset current number accumulator
    jmp parse_loop

finalize_sum:
    add dword [sum], ecx ; Add the last number
```

```
print_sum:
    ; Print the sum using printf
    push dword [sum]     ; Push the sum onto the stack
    push fmt_sum         ; Push the format string
    call printf          ; Print the sum
    add esp, 8           ; Clean up the stack

close_file:
    ; Close file
    mov eax, 6           ; sys_close
    mov ebx, edi         ; File descriptor
    int 0x80

exit:
    mov eax, 1           ; sys_exit
    xor ebx, ebx         ; Exit status 0
    int 0x80
```

c. Sum of randomInt100.txt

```
[zl51219@linux4 PrintSumProj]$ ./read
Sum: 4679
```

Sum of my own test file (number 1-20)

```
[zl51219@linux4 PrintSumProj]$ ./read
Sum: 210
```